

面向电影编程（MOVIES-OOP）

一、 系统介绍

“金融，工程，建筑，计算，固然很重要，那是我们生存的条件；但诗歌，浪漫，爱，是我们生而为人的原因。”电影作为当今最普遍的娱乐方式，它能够唤起情感共鸣、传递思想和文化，提供愉悦和放松的体验。无论是在艺术上、文化上还是娱乐上，电影都有着不可替代的地位和影响力。

项目“面向电影编程”以面向对象程序设计为基础，旨在通过 Java 语言的强大特性，为用户提供一个精确而个性化的电影推荐和查询系统。

该系统不同于传统的电影数据库，它采用面向对象的程序设计理念，将电影、演员、导演等各个元素视作对象，并通过对象之间的关联和继承，构建了一个完整且灵活的数据模型。用户可以根据自己的兴趣和喜好，通过多种方式快速查找和筛选电影，如电影名、导演、演员等，系统能够智能地推荐符合用户口味的电影作品。

通过面向对象的程序设计，系统实现了高内聚、低耦合的结构，使得功能模块之间的交互更加清晰和可维护。每个电影对象都包含丰富的信息，如片名、上映日期、评分、剧情简介等，用户可以在系统中轻松地获取这些详细信息，并通过用户界面友好的设计进行交互。

项目“面向电影编程”不仅仅是一个电影信息的存储和检索平台，更是一个通过现代程序设计理念实现的电影文化的推广和体验平台。它通过技术的力量，为用户提供了一个全新的电影探索和欣赏方式，帮助他们更深入地理解和享受电影艺术的无限魅力。

无论是研究电影的学者、电影爱好者还是寻找放松时光的普通观众，项目“面向电影编程”都能够为他们带来令人满意的电影体验，让每一个用户都能在这个平台上找到属于自己的电影世界。

二、 系统设计

1. 开发工具

1) MySQL

数据库管理系统，用于存储和管理数据。

2) Navicat Premium 16

数据库管理和开发工具，用于连接和管理 MySQL 数据库。

3) Java-SE17

Java 平台标准版 17，提供 Java 开发和运行环境。

4) Eclipse IDE 2024.3

集成开发环境，用于编写、调试和运行 Java 代码。

5) WindowBuilder

Eclipse 的插件，用于设计和构建 Java GUI（图形用户界面）。

2. 相关技术

1) Swing GUI（图形用户界面）

概述：使用 Java Swing 库构建用户界面，包括窗口、按钮、文本框等组件，提供用户友好的交互体验。

应用：在登录页面（LoginFrame）、注册页面（RegisterFrame）、用户主页面（UserMainFrame）、管理员主页面（AdminMainFrame）等处，利用 Swing 设计和实现了各种 GUI 组件。

2) JDBC (Java Database Connectivity)

概述：Java 程序与数据库 (MySQL) 连接的标准 API，用于执行 SQL 查询、更新和管理数据库。

应用：在 DAO (Data Access Object) 类中，例如 UserDao、AdminDao、MovieDao 等，通过 JDBC 与 MySQL 数据库进行交互，实现数据的持久化和操作。

3) MySQL 数据库

概述：开源的关系型数据库管理系统，用于存储和管理电影、用户、演员、导演等数据。

应用：项目中使用 MySQL 作为主要的数据存储介质，存储电影信息、用户信息、演员信息、导演信息等数据。

4) DAO 模式 (Data Access Object)

概述：设计模式之一，用于将应用程序与数据存储分离，通过 DAO 类提供对数据的访问操作接口。

应用：UserDao、AdminDao、MovieDao 等 DAO 类封装了对应的数据库操作，例如查询用户信息、添加电影、获取演员列表等。

5) 日期处理

概述：在 Java 中处理日期和时间的相关类和方法。

应用：在电影添加 (AdminMainFrame) 中，处理电影上映日期 (releaseDateField)、演员出生日期 (actorBirthDateField)、导演出生日期 (directorBirthDateField) 等输入的日期数据。

6) 异常处理

概述：在 Java 中处理可能发生的异常情况，保证程序的稳定性和可靠性。

应用：在用户输入验证、数据库操作、日期解析等过程中，通过 try-catch 块捕获并处理可能抛出的异常，例如 NumberFormatException 和 ParseException 等。

7) 设计模式

概述：在项目中合理应用设计模式，提高代码的重用性、可维护性和灵活性。

应用：可能应用了诸如工厂模式、观察者模式 (监听器模式) 等，尽量减少代码的耦合度，使得各部分功能更加独立和可扩展。

3. 系统结构

1) 数据库设计

数据库设计是系统的核心部分之一，它决定了数据存储的结构和查询效率。以下是项目的数据库表结构：

a) 电影类型表 (MovieTypes)：



The screenshot shows a database table with the following structure and data:

对象	movietypes @movies (0608) - 表
开始事务	文本 筛选 排序 导入
type_id	type_name
1	Action
2	Comedy
3	Drama
4	Sci-Fi
5	Horror
6	Romance
7	Thriller
8	Animation
9	Fantasy
10	Adventure

b) 地区表 (Regions):

对象 regions @movies (0608) - 表	
开始事务	文本 筛选 排序 导入
region_id	region_name
1	USA
2	UK
3	Canada
4	France
5	Germany
6	Australia
7	Japan
8	China
9	South Korea
10	India

c) 导演表 (Directors):

对象 directors @movies (0608) - 表		
开始事务	文本 筛选 排序 导入 导出 数据生成 创建	
director_id	director_name	birth_date
1	Christopher Nolan	1970-07-30
2	Quentin Tarantino	1963-03-27
3	Steven Spielberg	1946-12-18
4	Martin Scorsese	1942-11-17
5	James Cameron	1954-08-16
6	Peter Jackson	1961-10-31
7	Alfred Hitchcock	1899-08-13
8	Akira Kurosawa	1910-03-23
9	Hayao Miyazaki	1941-01-05
10	Spike Lee	1957-03-20

d) 演员表 (Actors):

对象 actors @movies (0608) - 表		
开始事务 文本 筛选 排序 导入 导出		
actor_id	actor_name	birth_date
1	Tom Hanks	1956-07-09
2	Leonardo DiCaprio	1974-11-11
3	Meryl Streep	1949-06-22
4	Brad Pitt	1963-12-18
5	Emma Stone	1988-11-06
6	Robert Downey Jr.	1965-04-04
7	Jennifer Lawrence	1990-08-15
8	Will Smith	1968-09-25
9	Scarlett Johansson	1984-11-22
10	Chris Evans	1981-06-13

e) 用户表 (Users):

对象 users @movies (0608) - 表		
开始事务 文本 筛选 排序 导入 导出		
user_id	username	password
1	user1	password1
2	user2	password2
3	user3	password3
4	user4	password4
5	user5	password5
6	user6	password6
7	user7	password7
8	user8	password8
9	user9	password9
10	user10	password10
123	xunxxxx	123

f) 管理员表 (Admins):

对象 admins @movies (0608) - 表			
开始事务 文本 筛选 排序 导入 导出 数据生成			
admin_id	username	password	
1	admin1	adminpass1	
2	admin2	adminpass2	
3	admin3	adminpass3	
4	admin4	adminpass4	
5	admin5	adminpass5	
6	admin6	adminpass6	
7	admin7	adminpass7	
8	admin8	adminpass8	
9	admin9	adminpass9	
10	admin10	adminpass10	

g) 电影表 (Movies):

对象 movies @movies (0608) - 表									
开始事务 文本 筛选 排序 导入 导出 数据生成 创建图表									
movie_id	title	release_date	duration	rating	plot_summary	type_id	region_id	director_id	
1	Inception	2010-07-16		148	8.8 A thief who steals corporate secrets th	4	1	1	1
2	Pulp Fiction	1994-10-14		154	8.9 The lives of two mob hitmen, a boxer,	7	1		2
3	Schindler's List	1993-12-15		195	8.9 In German-occupied Poland during W	3	1		3
4	The Godfathe	1972-03-24		175	9.2 The aging patriarch of an organized ci	3	1		4
5	Avatar	2009-12-18		162	7.8 A paraplegic marine dispatched to the	4	1		5
6	The Lord of tl	2003-12-17		201	8.9 Gandalf and Aragorn lead the World c	9	2		6
7	Psycho	1960-09-08		109	8.5 A Phoenix secretary embezzles \$40,000	5	1		7
8	Seven Samuri	1954-04-26		207	8.6 A poor village under attack by bandits	1	7		8
9	Spirited Away	2001-07-20		125	8.6 During her family's move to the subur	8	7		9
10	Black Panther	2018-02-16		134	7.3 T'Challa, heir to the hidden but advanc	1	1		10

h) 电影演员关系表 (MovieActors):

对象		movieactors @movies (0608) - 表	
开始事务		文本	筛选
		排序	导入
movie_id	actor_id		
3	1		
1	2		
6	2		
7	3		
2	4		
4	6		
5	7		
8	8		
9	9		
10	10		

i) 用户收藏表 (UserFavorites):

对象		userfavorites @movies (0608) - 表	
开始事务		文本	筛选
		排序	导入
		导出	数据生成
	movie_id	date_added	
1	1	2024-06-10 20:24:31	
2	3	2024-06-10 20:24:31	
3	4	2024-06-10 20:24:31	
4	6	2024-06-10 20:24:31	
5	8	2024-06-10 20:24:31	
6	10	2024-06-10 20:24:31	
7	2	2024-06-10 20:24:31	
8	5	2024-06-10 20:24:31	
9	7	2024-06-10 20:24:31	
10	9	2024-06-10 20:24:31	

j) 用户评分表 (UserRatings):

对象 userratings @movies (0608) - 表				
开始事务 文本 筛选 排序 导入 导出 数据生成 创建图表				
user_id	movie_id	rating	dateRated	
1	1	1	9 2024-06-10 20:24:32	
2	2	3	9.5 2024-06-10 20:24:32	
3	3	4	9.2 2024-06-10 20:24:32	
4	4	6	9.1 2024-06-10 20:24:32	
5	5	8	8.7 2024-06-10 20:24:32	
6	6	10	7.8 2024-06-10 20:24:32	
7	7	2	9.3 2024-06-10 20:24:32	
8	8	5	7.9 2024-06-10 20:24:32	
9	9	7	8.6 2024-06-10 20:24:32	
10	10	9	8.8 2024-06-10 20:24:32	

2) 模型类设计

以下是项目“面向电影编程”中主要类的设计，每个类都包含其相应的属性和方法，采用面向对象设计方法，使系统具有高内聚、低耦合的特点。

a) Movie 类:

属性:

- movieId (int): 电影 ID
- title (String): 电影标题
- releaseDate (Date): 上映日期
- duration (int): 电影时长 (分钟)
- rating (double): 评分
- plotSummary (String): 剧情简介
- type (MovieType): 电影类型
- region (Region): 制片地区

director (Director): 导演

actors (List<Actor>): 演员列表

方法:

构造函数: 初始化 Movie 对象

Getters 和 Setters: 访问和修改各属性的方法

toString(): 返回电影详细信息的字符串表示

setTypeId(int typeId): 根据类型 ID 设置电影类型

setRegionId(int regionId): 根据地区 ID 设置制片地区

setDirectorId(int directorId): 根据导演 ID 设置导演

b) Director 类:

属性:

directorId (int): 导演 ID

directorName (String): 导演 20

birthDate (Date): 出生日期

方法:

构造函数: 初始化 Director 对象

Getters 和 Setters: 访问和修改各属性的方法

c) Actor 类:

属性:

actorId (int): 演员 ID

actorName (String): 演员 20

birthDate (Date): 出生日期

方法:

构造函数: 初始化 Actor 对象

Getters 和 Setters: 访问和修改各属性的方法

d) User 类:

属性:

userId (int): 用户 ID

username (String): 用户名

password (String): 密码

favoriteMovies (List<Movie>): 收藏的电影列表

ratings (List<UserRating>): 用户评分列表

方法:

构造函数: 初始化 User 对象

Getters 和 Setters: 访问和修改各属性的方法

e) Admin 类:

属性:

adminId (int): 管理员 ID

username (String): 用户名

password (String): 密码

方法:

构造函数: 初始化 Admin 对象

Getters 和 Setters: 访问和修改各属性的方法

f) 其他类:

包括 MovieType、Region、MovieActor、UserFavorite、UserRating 等，每个类都包含其相应的属性和方法。

3) 数据访问类设计

每个 DAO 类负责一个特定实体类的数据操作，包括添加、

更新、删除和查询操作。这种设计将数据访问逻辑与业务逻辑分离，提高了代码的可维护性和扩展性。所有 DAO 类都继承自 BaseDao，共享基础的数据库连接管理方法，从而简化了数据库操作的实现。

a) BaseDao.java:

BaseDao 是所有 DAO 类的基类，负责管理与数据库的连接和关闭操作。它提供了以下两个方法：

getConnection(): 建立并返回一个数据库连接。如果连接失败，会打印堆栈跟踪信息。

closeConnection(Connection connection): 关闭给定的数据库连接。如果关闭失败，会打印堆栈跟踪信息。

b) ActorDao.java:

ActorDao 负责与 Actors 表进行交互，提供增删改查功能：

addActor(Actor actor): 添加一个新的演员记录。

updateActor(Actor actor): 更新现有演员的记录。

deleteActor(int actorId): 根据演员 ID 删除一条记录。

getActorById(int actorId): 根据演员 ID 获取演员详细信息。

getAllActors(): 获取所有演员的列表。

c) AdminDao.java:

AdminDao 负责与 Admins 表进行交互，提供增删改查功能：

addAdmin(Admin admin): 添加一个新的管理员记录。

updateAdmin(Admin admin): 更新现有管理员的记录。

deleteAdmin(int adminId): 根据管理员 ID 删除一条记录。

getAdminById(int adminId): 根据管理员 ID 获取管理员详细信息。

`getAllAdmins()`: 获取所有管理员的列表。

d) `DirectorDao.java`:

`DirectorDao` 负责与 `Directors` 表进行交互，提供增删改查功能：

`addDirector(Director director)`: 添加一个新的导演记录。

`updateDirector(Director director)`: 更新现有导演的记录。

`deleteDirector(int directorId)`: 根据导演 ID 删除一条记录。

`getDirectorById(int directorId)`: 根据导演 ID 获取导演详细信息。

`getAllDirectors()`: 获取所有导演的列表。

e) `MovieDao.java`:

`MovieDao` 负责与 `Movies` 表进行交互，提供增删改查功能：

`addMovie(Movie movie)`: 添加一个新的电影记录。

`updateMovie(Movie movie)`: 更新现有电影的记录。

`deleteMovie(int movieId)`: 根据电影 ID 删除一条记录。

`getMovieById(int movieId)`: 根据电影 ID 获取电影详细信息。

`getAllMovies()`: 获取所有电影的列表。

f) `UserDao.java`:

`UserDao` 负责与 `Users` 表进行交互，提供对用户信息的增删改查功能。主要方法包括：

`addUser(User user)`: 添加一个新的用户记录到数据库。

`updateUser(User user)`: 更新现有用户的记录。

`deleteUser(int userId)`: 根据用户 ID 删除用户记录。

`getUserById(int userId)`: 根据用户 ID 获取用户详细信息。

`getAllUsers()`: 获取所有用户的列表。

g) 其他 dao 类

包括 MovieActorDao、MovieTypeDao、RegionDao、UserFavoriteDao、UserRatingDao 等。

4) 工具类设计

DBUtil 和 StringUtil 是 com.xun.util 包中的两个重要工具类。DBUtil 简化了数据库连接的获取过程，集中管理数据库连接参数，而 StringUtil 则提供了常用的字符串处理方法。这些工具类提高了代码的可维护性和复用性，减少了重复代码，提升了开发效率。

a) DBUtil.java:

DBUtil 类负责管理与数据库的连接。它封装了获取数据库连接的逻辑，简化了 DAO 类中数据库操作的实现。

功能介绍：

URL, USER, PASSWORD: 这些常量定义了连接到数据库所需的 URL、用户名和密码。

getConnection(): 这个静态方法通过 DriverManager.getConnection() 创建并返回一个新的数据库连接。它使用上面定义的常量来连接到特定的数据库。如果连接失败，会抛出 SQLException。

优点：

集中管理数据库连接信息，便于维护和修改。

提供了简洁的方法来获取数据库连接，减少了重复代码。

b) StringUtil.java:

StringUtil 类提供了一些用于字符串处理的常用方法。

功能介绍:

isEmpty(String str): 这个静态方法检查给定的字符串是否为空或仅包含空白字符。它首先检查字符串是否为 null, 然后使用 trim() 方法去除首尾空白字符, 再检查结果是否为空字符串。

优点:

提供了一个常用的字符串检查方法, 简化了空字符串判断的代码。

代码可读性高, 易于理解和使用。

5) 界面类设计

界面设计: 各个界面都采用了简洁明了的布局和组件排列, 方便用户和管理员使用。

功能实现: 通过 DAO 层与数据库交互, 实现了添加、查找、登录和注册功能。

输入验证: 确保用户输入的数据格式正确, 提高了程序的健壮性。

用户体验: 通过合理的界面设计和功能实现, 提供了友好的用户体验。

a) AdminMainFrame.java:

AdminMainFrame 是管理员主界面, 允许管理员添加电影、演员和导演。

设计细节:

布局: 使用 JTabbedPane 来创建三个标签页: 添加电影、添加演员、添加导演。

组件: 每个标签页都有相应的输入字段和按钮, 允许管理员输入数据并提交。

添加电影面板：包含电影 ID、标题、上映日期、时长、评分、剧情简介、类型 ID、地区 ID 和导演 ID 的输入字段，及一个提交按钮。

添加演员面板：包含演员 ID、演员名字和出生日期的输入字段，及一个提交按钮。

添加导演面板：包含导演 ID、导演名字和出生日期的输入字段，及一个提交按钮。

功能：每个面板的提交按钮都绑定了相应的事件监听器。当按钮被点击时，收集输入数据并调用 `movieDao`, `actorDao`, `directorDao` 分别添加电影、演员和导演。

输入验证：在添加数据前对输入进行验证，确保数据格式正确，例如数字类型的字段必须是整数或浮点数，日期字段必须是有效的日期格式。

b) `LoginFrame.java`:

`LoginFrame` 是用户和管理员的登录界面。

设计细节：

布局：使用 `JPanel` 和布局管理器来排列组件。

组件：包括用户 ID、密码输入字段，用户类型选择框（用户或管理员），登录按钮和注册按钮。

功能：

登录：根据用户选择的类型（用户或管理员），检查输入的 ID 和密码是否匹配数据库中的记录。

注册：点击注册按钮时，打开注册界面 `RegisterFrame`。

输入验证：检查 ID 和密码是否为空，验证登录时根据选择的用户类型查询对应的数据库表。

c) `RegisterFrame.java`:

`RegisterFrame` 是用户注册界面。

设计细节：

布局：使用 JPanel 和布局管理器来排列组件。

组件：包括用户 ID、用户名、密码输入字段和注册按钮。

功能：

注册：点击注册按钮时，收集输入数据并调用 userDao 添加新用户到数据库。

输入验证：检查所有输入字段是否为空，ID 必须是有效的整数格式。

d) UserMainFrame.java:

UserMainFrame 是用户主界面，允许用户查找电影和查看推荐电影。

设计细节：

布局：使用 JTabbedPane 来创建两个标签页：查找电影和推荐电影。

组件：

查找电影面板：包含演员名字、导演名字、电影标题和年份的输入字段及一个查找按钮，显示搜索结果的文本区域。

推荐电影面板：显示每日推荐和近期热门推荐的文本区域。

功能：

查找电影：根据输入的条件调用 movieDao.searchMovies 搜索匹配的电影，并在结果区域显示。

推 荐 电 影 ： 调 用 movieDao.getDailyRecommendation 和 movieDao.getRecentHotRecommendations 显示每日推荐和近期热门推荐电影。

4. 模块关系

1) 登录页面 (LoginFrame)

用户通过登录页面输入用户名和密码进行身份验证，成功后进入主页面。

2) 主页面

根据用户类型显示不同的主页面内容，包括普通用户主页面和管理员主页面。

5. 各模块功能说明

1) DAO (数据访问对象类): 负责与数据库的交互操作。

ActorDao.java: 处理与演员相关的数据库操作。

AdminDao.java: 处理与管理相关的数据库操作。

BaseDao.java: 提供通用的数据库操作方法。

DirectorDao.java: 处理与导演相关的数据库操作。

MovieActorDao.java: 处理电影与演员关联的数据库操作。

MovieDao.java: 处理与电影相关的数据库操作。

MovieTypeDao.java: 处理与电影类型相关的数据库操作。

RegionDao.java: 处理与地区相关的数据库操作。

UserDao.java: 处理与用户相关的数据库操作。

UserFavoriteDao.java: 处理用户收藏电影的数据库操作。

UserRatingDao.java: 处理用户对电影评分的数据库操作。

2) Model (数据模型类): 存放数据的模型类和构造输出。

Actor.java: 演员类, 包含演员的属性和方法。

Admin.java: 管理员类, 包含管理员的属性和方法。

Director.java: 导演类, 包含导演的属性和方法。

Movie.java: 电影类, 包含电影的属性和方法。

MovieActor.java: 电影和演员关联类。

MovieType.java: 电影类型类。

Region.java: 地区类。

User.java: 用户类, 包含用户的属性和方法。

UserFavorite.java: 用户收藏类。

UserRating.java: 用户评分类。

3) Util (工具类): 一些工具性和辅助性的操作文件。

DBUtil.java: 提供数据库连接和关闭连接的方法。

StringUtil.java: 提供字符串相关的实用方法, 如空值检查等。

4) View (视图类): 视图类文件, 负责界面的展示和交互。

AdminMainFrame.java: 管理员主界面, 提供添加电影, 添加导演, 添加演员等功能。

LoginFrame.java: 登录界面, 提供用户和管理员登录功能。

RegisterFrame.java: 注册界面, 提供新用户注册功能。

5) 参考库

项目中引用了 MySQL 数据库驱动库: 该驱动库用于 Java 应用程序与 MySQL 数据库的连接和交互

三、 系统类结构（UML 图）

1. Actor 类

```
+-----+
|      Actor      |
+-----+
| - actorId: int   |
| - actorName: String|
| - birthDate: Date |
+-----+
| + Actor()        |
| + getActorId(): int|
| + setActorId(actorId: int): void|
| + getActorName(): String|
| + setActorName(actorName: String): void|
| + getBirthDate(): Date|
| + setBirthDate(birthDate: Date): void|
+-----+
```

2. Admin 类

```
+-----+
|      Admin      |
+-----+
| - adminId: int   |
| - username: String|
| - password: String|
+-----+
| + Admin()        |
| + getAdminId(): int|
| + setAdminId(adminId: int): void|
| + getUsername(): String|
| + setUsername(username: String): void|
| + getPassword(): String|
| + setPassword(password: String): void|
+-----+
```

3. Director 类

```
+-----+
|      Director    |
+-----+
| - directorId: int |
| - directorName: String |
| - birthDate: Date  |
+-----+
| + Director()       |
| + getDirectorId(): int |
| + setDirectorId(directorId: int): void |
| + getDirectorName(): String |
| + setDirectorName(directorName: String): void |
| + getBirthDate(): Date |
| + setBirthDate(birthDate: Date): void |
+-----+
```

4. Movie 类

```
+-----+
|      Movie      |
+-----+
| - movieId: int   |
| - title: String  |
| - releaseDate: Date|
| - duration: int  |
| - rating: double |
| - plotSummary: String |
| - type: MovieType |
| - region: Region  |
| - director: Director |
| - actors: List<Actor>|
+-----+
| + Movie()        |
| + getMovieId(): int|
| + setMovieId(movieId: int): void |
| + getTitle(): String|
| + setTitle(title: String): void |
| + getReleaseDate(): Date|
| + setReleaseDate(releaseDate: Date): void |
| + getDuration(): int|
| + setDuration(duration: int): void |
| + getRating(): double|
| + setRating(rating: double): void |
| + getPlotSummary(): String|
| + setPlotSummary(plotSummary: String): void |
| + getType(): MovieType|
| + setType(type: MovieType): void |
| + getRegion(): Region|
| + setRegion(region: Region): void |
| + getDirector(): Director|
| + setDirector(director: Director): void |
| + getActors(): List<Actor>|
| + setActors(actors: List<Actor>): void |
| + toString(): String|
| + setTypeId(typeId: int): void |
| + setRegionId(regionId: int): void |
| + setDirectorId(directorId: int): void |
+-----+
```

5. MovieActor 类

```
+-----+
|   MovieActor   |
+-----+
| - movieId: int  |
| - actorId: int  |
+-----+
| + MovieActor()  |
| + getMovieId(): int|
| + setMovieId(movieId: int): void |
| + getActorId(): int|
| + setActorId(actorId: int): void |
+-----+
```

6. MovieType 类

```
+-----+
|   MovieType    |
+-----+
| - typeId: int   |
| - typeName: String|
+-----+
| + MovieType()   |
| + getTypeId(): int|
| + setTypeId(typeId: int): void |
| + getTypeName(): String|
| + setTypeName(typeName: String): void |
+-----+
```

7. Region 类

```
+-----+
|      Region      |
+-----+
| - regionId: int   |
| - regionName: String|
+-----+
| + Region()        |
| + getRegionId(): int|
| + setRegionId(regionId: int): void |
| + getRegionName(): String|
| + setRegionName(regionName: String): void |
+-----+
```


8. User 类

```
+-----+
|      User      |
+-----+
| - userId: int   |
| - username: String|
| - password: String|
| - favoriteMovies: List<Movie>|
| - ratings: List<UserRating>|
+-----+
| + User()        |
| + User(userId: int, username: String, password: String)|
| + getUserId(): int|
| + setUserId(userId: int): void |
| + getUsername(): String|
| + setUsername(username: String): void |
| + getPassword(): String|
| + setPassword(password: String): void |
| + getFavoriteMovies(): List<Movie>|
| + setFavoriteMovies(favoriteMovies: List<Movie>): void |
| + getRatings(): List<UserRating>|
| + setRatings(ratings: List<UserRating>): void |
+-----+
```

9. UserFavorite 类

```
+-----+
|  UserFavorite  |
+-----+
| - userId: int   |
| - movieId: int  |
| - dateAdded: Date |
+-----+
| + UserFavorite() |
| + getUserId(): int |
| + setUserId(userId: int): void |
| + getMovieId(): int |
| + setMovieId(movieId: int): void |
| + getDateAdded(): Date |
| + setDateAdded(dateAdded: Date): void |
+-----+
```

10. UserRating 类

```
+-----+
|   UserRating   |
+-----+
| - userId: int   |
| - movieId: int  |
| - rating: double|
| - dateRated: Date|
+-----+
| + UserRating()  |
| + getUserId(): int|
| + setUserId(userId: int): void |
| + getMovieId(): int|
| + setMovieId(movieId: int): void |
| + getRating(): double|
| + setRating(rating: double): void |
| + getDateRated(): Date|
| + setDateRated(dateRated: Date): void |
+-----+
```

11. BaseDao 类

```
+-----+
|   BaseDao      |
+-----+
| - URL: String   |
| - USER: String  |
| - PASSWORD: String |
+-----+
| + getConnection(): Connection |
| + closeConnection(connection: Connection): void |
+-----+
```

12. ActorDao 类

```
-----  
|      ActorDao      |  
-----  
| - URL: String      |  
| - USER: String     |  
| - PASSWORD: String |  
-----  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addActor(actor: Actor): void |  
| + updateActor(actor: Actor): void |  
| + deleteActor(actorId: int): void |  
| + getActorById(actorId: int): Actor |  
| + getAllActors(): List<Actor> |  
-----
```

13. AdminDao 类

```
-----  
|      AdminDao      |  
-----  
| - URL: String      |  
| - USER: String     |  
| - PASSWORD: String |  
-----  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addAdmin(admin: Admin): void |  
| + updateAdmin(admin: Admin): void |  
| + deleteAdmin(adminId: int): void |  
| + getAdminById(adminId: int): Admin |  
| + getAllAdmins(): List<Admin> |  
-----
```

14. DirectorDao 类

```
-----  
|      DirectorDao      |  
-----  
| - URL: String          |  
| - USER: String         |  
| - PASSWORD: String     |  
-----  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addDirector(director: Director): void |  
| + updateDirector(director: Director): void |  
| + deleteDirector(directorId: int): void |  
| + getDirectorById(directorId: int): Director |  
| + getAllDirectors(): List<Director> |  
-----
```

15. MovieDao 类

```
-----  
|      MovieDao          |  
-----  
| - URL: String          |  
| - USER: String         |  
| - PASSWORD: String     |  
-----  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addMovie(movie: Movie): void |  
| + updateMovie(movie: Movie): void |  
| + deleteMovie(movieId: int): void |  
| + getMovieById(movieId: int): Movie |  
| + getAllMovies(): List<Movie> |  
| + searchMovies(actor: String, director: String, title: String, year: int): List<Movie> |  
| + searchMoviesByYear(year: int): List<Movie> |  
| + getDailyRecommendation(): List<Movie> |  
| + getRecentHotRecommendations(): List<Movie> |  
| - extractMovieFromResultSet(rs: ResultSet): Movie |  
| - getActorsByMovieId(movieId: int): List<Actor> |  
-----
```

16. MovieActorDao 类

```
-----  
|      MovieActorDao      |  
|-----|  
| - URL: String           |  
| - USER: String          |  
| - PASSWORD: String       |  
|-----|  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addMovieActor(movieActor: MovieActor): void |  
| + deleteMovieActor(movieId: int, actorId: int): void |  
| + getActorsByMovieId(movieId: int): List<MovieActor> |  
| + getMoviesByActorId(actorId: int): List<MovieActor> |  
|-----|
```

17. MovieTypeDao 类

```
-----  
|      MovieTypeDao      |  
|-----|  
| - URL: String           |  
| - USER: String          |  
| - PASSWORD: String       |  
|-----|  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addMovieType(movieType: MovieType): void |  
| + updateMovieType(movieType: MovieType): void |  
| + deleteMovieType(typeId: int): void |  
| + getMovieTypeById(typeId: int): MovieType |  
| + getAllMovieTypes(): List<MovieType> |  
|-----|
```

18. RegionDao 类

```
-----  
|           RegionDao           |  
|-----|  
| - URL: String                 |  
| - USER: String               |  
| - PASSWORD: String           |  
|-----|  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addRegion(region: Region): void |  
| + updateRegion(region: Region): void |  
| + deleteRegion(regionId: int): void |  
| + getRegionById(regionId: int): Region |  
| + getAllRegions(): List<Region> |  
|-----|
```

19. UserDao 类

```
-----  
|           UserDao           |  
|-----|  
| - URL: String                 |  
| - USER: String               |  
| - PASSWORD: String           |  
|-----|  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addUser(userId: int, username: String, password: String): void |  
| + updateUser(user: User): void |  
| + deleteUser(userId: int): void |  
| + getUserById(userId: int): User |  
| + getAllUsers(): List<User> |  
|-----|
```

20. UserFavoriteDao 类

```
-----  
|      UserFavoriteDao      |  
-----  
| + URL: String             |  
| + USER: String            |  
| + PASSWORD: String         |  
-----  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addUserFavorite(userFavorite: UserFavorite): void |  
| + deleteUserFavorite(userId: int, movieId: int): void |  
| + getFavoritesByUserId(userId: int): List<UserFavorite> |  
| + getUsersByMovieId(movieId: int): List<UserFavorite> |  
-----
```

21. UserRatingDao 类

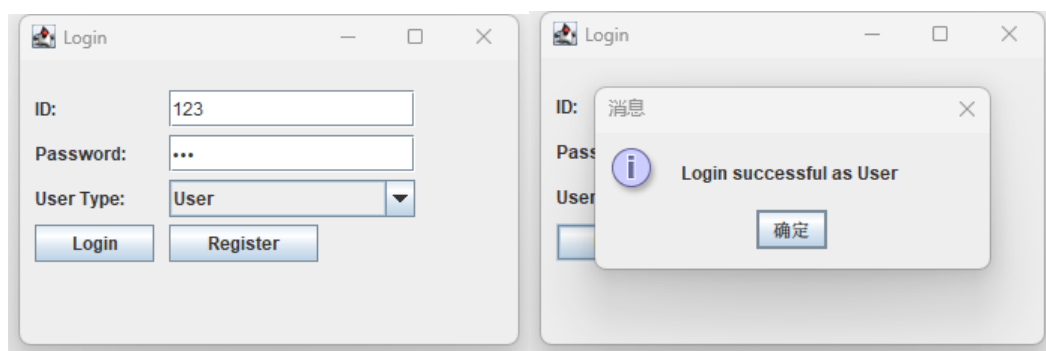
```
-----  
|      UserRatingDao      |  
-----  
| + URL: String             |  
| + USER: String            |  
| + PASSWORD: String         |  
-----  
| + getConnection(): Connection |  
| + closeConnection(connection: Connection): void |  
| + addUserRating(userRating: UserRating): void |  
| + updateUserRating(userRating: UserRating): void |  
| + deleteUserRating(userId: int, movieId: int): void |  
| + getRatingsByUserId(userId: int): List<UserRating> |  
| + getRatingsByMovieId(movieId: int): List<UserRating> |  
-----
```


四、 系统界面运行截图及使用说明

通过界面的交互功能，用户和管理员可以轻松地进行登录、注册、电影搜索、电影推荐以及电影、演员和导演信息的添加和管理。

1. 登录页面（LoginFrame）

登录页面是用户进入系统的入口，通过输入用户名和密码进行身份验证。如果登录成功，用户将被重定向到相应的主页面（用户主页面或管理员主页面）。



1) 界面组件

ID 输入框：用户输入其唯一标识 ID。

密码输入框：用户输入密码。

用户类型选择框：用户选择自己的类型（User 或 Admin）。

登录按钮：点击后，系统会验证输入的 ID 和密码。

注册按钮：新用户可以通过点击此按钮进入注册页面。

2) 使用步骤

输入 ID 和密码。

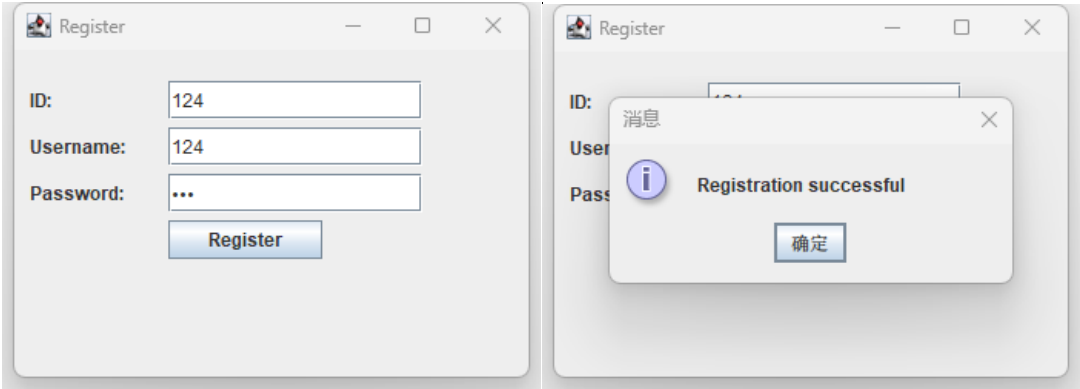
从下拉框中选择用户类型。

点击“登录”按钮进行登录。如果验证通过，将进入相应的主页面。

如果还没有账户，点击“注册”按钮进入注册页面进行新用户注册。

2. 注册页面 (RegisterFrame)

注册页面允许新用户创建账户。用户需要填写 ID、用户名和密码，成功注册后可以使用这些信息进行登录。



user_id	username	password
1	user1	password1
2	user2	password2
3	user3	password3
4	user4	password4
5	user5	password5
6	user6	password6
7	user7	password7
8	user8	password8
9	user9	password9
10	user10	password10
123	xunxxxx	123
124	124	124

1) 界面组件

- ID 输入框：用户输入其唯一标识 ID。
- 用户名输入框：用户输入自己的用户名。
- 密码输入框：用户设置密码。
- 注册按钮：点击后，系统会将用户信息保存到数据库中。

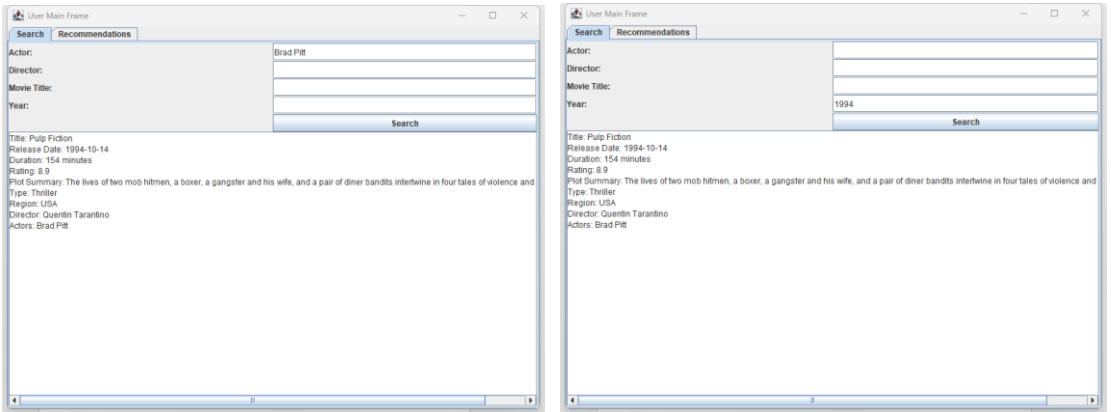
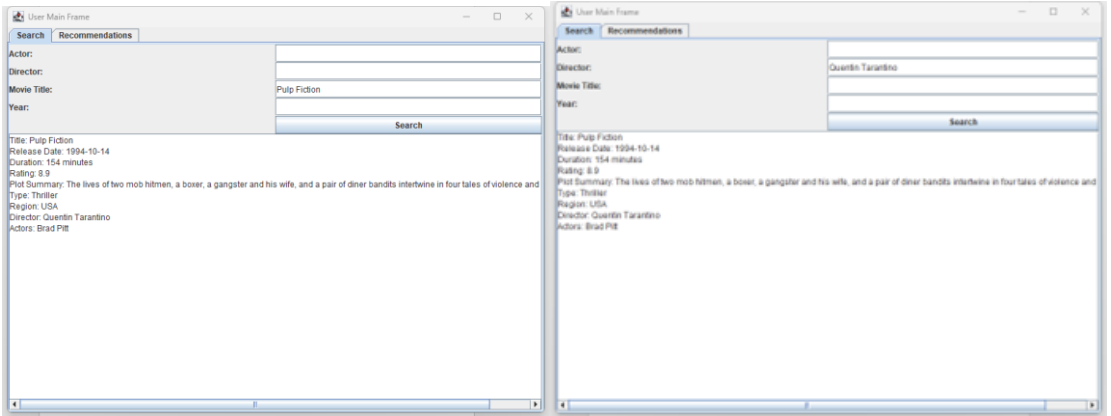
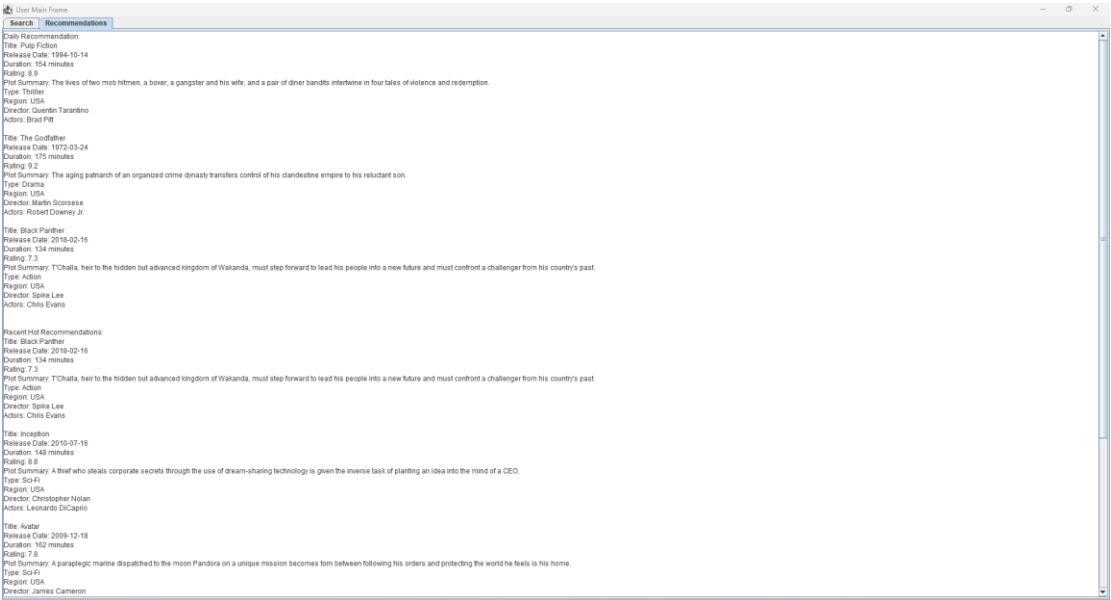
2) 使用步骤

输入 ID、用户名和密码。

点击“注册”按钮进行注册。
注册成功后，用户可以使用新注册的 ID 和密码在登录页面进行登录。

3. 用户主页面（UserMainFrame）

用户主页面提供电影搜索和推荐功能。



1) 界面组件

- 查找标签页：
 - 演员输入框：输入演员名字进行搜索。
 - 导演输入框：输入导演名字进行搜索。
 - 电影标题输入框：输入电影标题进行搜索。
 - 年份输入框：输入年份进行搜索。
 - 搜索按钮：点击后进行电影搜索。
 - 搜索结果区域：显示搜索结果。
- 推荐标签页：
 - 每日推荐：显示系统每日推荐的电影。
 - 近期热门推荐：显示系统近期热门推荐的电影。

2) 使用步骤

- 在“查找”标签页中，填写搜索条件（演员、导演、电影标题、年份）。
- 点击“搜索”按钮查看搜索结果。
- 在“推荐”标签页中查看系统推荐的电影。

4. 管理员主页面（AdminMainFrame）

管理员主页面提供添加电影、演员和导演的功能。

Admin Main Frame

添加电影 添加演员 添加导演

演员ID:

77

演员名字:

出生日期 (yyyy-mm-dd):

2003-11-17

添加演员

消息

演员添加成功!

确定

信息	摘要	结果 1	剖析	状态
actor_id	actor_name	birth_date		
1	Tom Hanks	1956-07-09		
2	Leonardo DiCaprio	1974-11-11		
3	Meryl Streep	1949-06-22		
4	Brad Pitt	1963-12-18		
5	Emma Stone	1988-11-06		
6	Robert Downey Jr.	1965-04-04		
7	Jennifer Lawrence	1990-08-15		
8	Will Smith	1968-09-25		
9	Scarlett Johansson	1984-11-22		
10	Chris Evans	1981-06-13		
77	XUNXXX	2003-11-17		

Admin Main Frame

添加电影 添加演员 添加导演

导演ID: 78

导演名字:

出生日期 (yyyy-mm-dd): 2003-11-17

添加导演

消息: 导演添加成功!

信息	摘要	结果 1	剖析	状态
director_id	director_name	birth_date		
1	Christopher Nolan	1970-07-30		
2	Quentin Tarantino	1963-03-27		
3	Steven Spielberg	1946-12-18		
4	Martin Scorsese	1942-11-17		
5	James Cameron	1954-08-16		
6	Peter Jackson	1961-10-31		
7	Alfred Hitchcock	1899-08-13		
8	Akira Kurosawa	1910-03-23		
9	Hayao Miyazaki	1941-01-05		
10	Spike Lee	1957-03-20		
78	XUN	2003-11-17		

Admin Main Frame

添加电影 添加演员 添加导演

电影ID: 79

标题: HELLO

上映日期 (yyyy-mm-dd):

时长 (分钟):

评分:

剧情介绍:

类型ID: 9

地区ID: 8

导演ID: 78

添加电影

消息: 电影添加成功!

信息	摘要	结果 1	剖析	状态					
movie_id	title	release_date	duration	rating	plot_summary	type_id	region_id	director_id	
1	Inception	2010-07-16	148	8.8	A thief who steals corpora	4	1	1	
2	Pulp Fiction	1994-10-14	154	8.9	The lives of two mob hittr	7	1	2	
3	Schindler's	1993-12-15	195	8.9	In German-occupied Pola	3	1	3	
4	The Godfat!	1972-03-24	175	9.2	The aging patriarch of an	3	1	4	
5	Avatar	2009-12-18	162	7.8	A paraplegic marine disp	4	1	5	
6	The Lord of	2003-12-17	201	8.9	Gandalf and Aragorn leac	9	2	6	
7	Psycho	1960-09-08	109	8.5	A Phoenix secretary embx	5	1	7	
8	Seven Sami	1954-04-26	207	8.6	A poor village under attai	1	7	8	
9	Spirited Aw	2001-07-20	125	8.6	During her family's move	8	7	9	
10	Black Panth	2018-02-16	134	7.3	T'Challa, heir to the hide	1	1	10	
79	HELLO	2024-06-13	100	9.9	A BOY	9	8	78	

1) 界面组件

添加电影面板:

电影 ID 输入框: 输入电影 ID。

标题输入框: 输入电影标题。

上映日期输入框: 输入上映日期 (格式为 yyyy-mm-dd)。

时长输入框: 输入电影时长 (分钟)。

评分输入框: 输入电影评分。

剧情简介输入框: 输入电影的剧情简介。

类型 ID 输入框: 输入电影类型 ID。

地区 ID 输入框: 输入电影地区 ID。

导演 ID 输入框: 输入导演 ID。

添加按钮: 点击后添加电影信息到数据库。

添加演员面板:

演员 ID 输入框: 输入演员 ID。

演员名字输入框: 输入演员名字。

出生日期输入框: 输入演员出生日期 (格式为 yyyy-mm-dd)。

添加按钮: 点击后添加演员信息到数据库。

添加导演面板:

导演 ID 输入框: 输入导演 ID。

导演名字输入框: 输入导演名字。

出生日期输入框: 输入导演出生日期 (格式为 yyyy-mm-dd)。

添加按钮: 点击后添加导演信息到数据库。

2) 使用步骤

选择相应的标签页（添加电影、添加演员、添加导演）。

填写需要添加的信息。

点击“添加”按钮将信息保存到数据库。

五、 核心代码说明

1. 数据库连接工具类 - DBUtil.java

```
package com.xun.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * 数据库连接工具类。
 */
public class DBUtil {
    // 数据库连接的 URL
    private static final String URL = "jdbc:mysql://localhost:3306/movies";

    // 数据库用户名
    private static final String USER = "root";

    // 数据库密码
    private static final String PASSWORD = "2003David";

    /**
     * 获取数据库连接对象。
     *
     * @return 表示数据库连接的 Connection 对象
     * @throws SQLException 如果发生数据库访问错误
     */
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

这个类的核心代码提供了一个数据库连接工具 DBUtil，包含以下关键点：
URL、USER 和 PASSWORD 常量：定义了连接到 MySQL 数据库所需的 URL、用户名和密码。

getConnection 方法：静态方法，用于返回一个 Connection 对象，该对象表示

与数据库的连接。方法使用 `DriverManager.getConnection` 方法通过提供的 URL、用户名和密码来获取数据库连接。

2. 数据访问对象类 - MovieDao.java

```
public List<Movie> getDailyRecommendation() {
    // 创建一个空的电影列表
    List<Movie> movies = new ArrayList<>();

    // SQL 查询语句，选择电影及其附加详细信息
    String sql = "SELECT m.*, t.type_name, r.region_name, d.director_name " +
        "FROM Movies m " +
        "LEFT JOIN MovieTypes t ON m.type_id = t.type_id " +
        "LEFT JOIN Regions r ON m.region_id = r.region_id " +
        "LEFT JOIN Directors d ON m.director_id = d.director_id " +
        "ORDER BY RAND() LIMIT 3";

    try (Connection connection = getConnection();
        // 准备执行 SQL 语句的预编译语句
        PreparedStatement statement = connection.prepareStatement(sql)) {

        // 执行查询并获取结果集
        ResultSet resultSet = statement.executeQuery();

        // 遍历结果集并将每行数据转换为电影对象，加入到电影列表中
        while (resultSet.next()) {
            movies.add(extractMovieFromResultSet(resultSet)); // 从结果集中提取电影详细信息
        }
    } catch (SQLException e) {
        e.printStackTrace(); // 处理 SQL 异常，打印堆栈跟踪
    }

    // 返回填充好的电影列表
    return movies;
}

public List<Movie> getRecentHotRecommendations() {
    // 创建一个空的电影列表
    List<Movie> movies = new ArrayList<>();

    // SQL 查询语句，选择最近热门推荐的电影及其附加详细信息
    String sql = "SELECT m.*, t.type_name, r.region_name, d.director_name " +
        "FROM Movies m " +
```

```

        "LEFT JOIN MovieTypes t ON m.type_id = t.type_id " +
        "LEFT JOIN Regions r ON m.region_id = r.region_id " +
        "LEFT JOIN Directors d ON m.director_id = d.director_id " +
        "ORDER BY m.release_date DESC LIMIT 5";

try (Connection connection = getConnection();
    // 准备执行 SQL 语句的预编译语句
    PreparedStatement statement = connection.prepareStatement(sql)) {

    // 执行查询并获取结果集
    ResultSet resultSet = statement.executeQuery();

    // 遍历结果集并将每行数据转换为电影对象，加入到电影列表中
    while (resultSet.next()) {
        movies.add(extractMovieFromResultSet(resultSet)); // 从结果集中提取电影详细信息
    }
} catch (SQLException e) {
    e.printStackTrace(); // 处理 SQL 异常，打印堆栈跟踪
}

// 返回填充好的最近热门推荐电影列表
return movies;
}

```

3. 用户界面类 - UserMainFrame.java

```

package com.xun.view;

import com.xun.dao.MovieDao;
import com.xun.model.Movie;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.List;

// 用户主界面类，继承自 JFrame
public class UserMainFrame extends JFrame {

    private MovieDao movieDao = new MovieDao(); // 创建电影数据访问对象

    // 构造方法
    public UserMainFrame() {

```



```

setTitle("User Main Frame"); // 设置窗口标题
setSize(800, 600); // 设置窗口大小
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 设置窗口关闭操作为退出程序
setLocationRelativeTo(null); // 设置窗口居中显示

JTabbedPane tabbedPane = new JTabbedPane(); // 创建选项卡面板

// 创建查找板块
JPanel searchPanel = new JPanel(new BorderLayout()); // 创建查找面板，使用边界布局
tabbedPane.addTab("Search", searchPanel); // 在选项卡面板中添加查找选项卡

JPanel searchFormPanel = new JPanel(new GridLayout(5, 2)); // 创建查找表单面板，使用
网格布局
searchPanel.add(searchFormPanel, BorderLayout.NORTH); // 将查找表单面板添加到查找面
板的北部（上方）

// 添加 Actor 字段及其文本框
searchFormPanel.add(new JLabel("Actor:"));
JTextField actorField = new JTextField();
searchFormPanel.add(actorField);

// 添加 Director 字段及其文本框
searchFormPanel.add(new JLabel("Director:"));
JTextField directorField = new JTextField();
searchFormPanel.add(directorField);

// 添加 Movie Title 字段及其文本框
searchFormPanel.add(new JLabel("Movie Title:"));
JTextField titleField = new JTextField();
searchFormPanel.add(titleField);

// 添加 Year 字段及其文本框
searchFormPanel.add(new JLabel("Year:"));
JTextField yearField = new JTextField();
searchFormPanel.add(yearField);

JButton searchButton = new JButton("Search"); // 创建“Search”按钮
searchFormPanel.add(new JLabel("")); // 添加一个空的标签，用于占位
searchFormPanel.add(searchButton); // 将“Search”按钮添加到查找表单面板

JTextArea searchResultArea = new JTextArea(); // 创建文本域用于显示搜索结果
searchPanel.add(new JScrollPane(searchResultArea), BorderLayout.CENTER); // 将搜索结
果文本域添加到查找面板的中心

```

```

// 添加搜索按钮的动作监听器
searchButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String actor = actorField.getText().trim(); // 获取 Actor 文本框的文本，并去除
        首尾空白

        String director = directorField.getText().trim(); // 获取 Director 文本框的文
        本，并去除首尾空白

        String title = titleField.getText().trim(); // 获取 Movie Title 文本框的文本，
        并去除首尾空白

        int year = 0;
        try {
            if (!yearField.getText().isEmpty()) { // 如果 Year 文本框不为空
                year = Integer.parseInt(yearField.getText().trim()); // 尝试将 Year 文
                本框的文本转换为整数
            }
        } catch (NumberFormatException ex) { // 捕获数字格式异常
            JOptionPane.showMessageDialog(UserMainFrame.this, "Invalid year format.
            Use a valid year."); // 显示错误消息对话框

            return;
        }

        // 调用电影数据访问对象的搜索电影方法，获取符合条件的电影列表
        List<Movie> movies = movieDao.searchMovies(actor.isEmpty() ? null : actor,
                                                    director.isEmpty() ? null :
            director,

                                                    title.isEmpty() ? null : title,
                                                    year);

        searchResultArea.setText(""); // 清空先前的搜索结果

        if (movies.isEmpty()) { // 如果电影列表为空
            searchResultArea.append("No movies found matching the search criteria.");
            // 在搜索结果文本域中显示未找到电影的消息
        } else {
            for (Movie movie : movies) { // 遍历电影列表
                searchResultArea.append(movie.toString() + "\n"); // 在搜索结果文本
                域中显示每部电影的详细信息
            }
        }
    }
});

```

```

        // 创建推荐板块
        JPanel recommendationPanel = new JPanel(new BorderLayout()); // 创建推荐面板，使用边
        界布局
        tabbedPane.addTab("Recommendations", recommendationPanel); // 在选项卡面板中添加推荐
        选项卡

        JTextArea recommendationArea = new JTextArea(); // 创建文本域用于显示推荐结果
        recommendationPanel.add(new JScrollPane(recommendationArea), BorderLayout.CENTER); //
        将推荐结果文本域添加到推荐面板的中心

        // 显示每日推荐和近期热门推荐
        recommendationArea.append("Daily Recommendation:\n"); // 显示每日推荐的标签
        List<Movie> dailyRecommendation = movieDao.getDailyRecommendation(); // 获取每日推荐
        电影列表
        for (Movie movie : dailyRecommendation) { // 遍历每日推荐电影列表
            recommendationArea.append(movie.toString() + "\n"); // 在推荐结果文本域中显示每
            部电影的详细信息
        }

        recommendationArea.append("\nRecent Hot Recommendations:\n"); // 显示近期热门推荐的
        标签
        List<Movie> recentHotRecommendations = movieDao.getRecentHotRecommendations(); // 获
        取近期热门推荐电影列表
        for (Movie movie : recentHotRecommendations) { // 遍历近期热门推荐电影列表
            recommendationArea.append(movie.toString() + "\n"); // 在推荐结果文本域中显示每
            部电影的详细信息
        }

        getContentPane().add(tabbedPane); // 将选项卡面板添加到窗口内容面板
    }

    // 主方法，程序入口
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new UserMainFrame().setVisible(true); // 创建用户主界面对象并显示窗口
            }
        });
    }
}

```

4. 实体模型类 - Movies.java

```
package com.xun.model;

import java.util.Date;
import java.util.List;

// 电影类
public class Movie {
    // 属性
    private int movieId; // 电影 ID
    private String title; // 标题
    private Date releaseDate; // 发布日期
    private int duration; // 持续时间（分钟）
    private double rating; // 评分
    private String plotSummary; // 情节摘要
    private MovieType type; // 电影类型
    private Region region; // 地区
    private Director director; // 导演
    private List<Actor> actors; // 演员列表

    // 构造方法
    public Movie() {}

    // Getter 和 Setter 方法
    // 获取电影 ID
    public int getMovieId() {
        return movieId;
    }

    // 设置电影 ID
    public void setMovieId(int movieId) {
        this.movieId = movieId;
    }

    // 获取电影标题
    public String getTitle() {
        return title;
    }

    // 设置电影标题
    public void setTitle(String title) {
        this.title = title;
    }
}
```

```
// 获取电影发布日期
public Date getReleaseDate() {
    return releaseDate;
}

// 设置电影发布日期
public void setReleaseDate(Date releaseDate) {
    this.releaseDate = releaseDate;
}

// 获取电影持续时间（分钟）
public int getDuration() {
    return duration;
}

// 设置电影持续时间（分钟）
public void setDuration(int duration) {
    this.duration = duration;
}

// 获取电影评分
public double getRating() {
    return rating;
}

// 设置电影评分
public void setRating(double rating) {
    this.rating = rating;
}

// 获取电影情节摘要
public String getPlotSummary() {
    return plotSummary;
}

// 设置电影情节摘要
public void setPlotSummary(String plotSummary) {
    this.plotSummary = plotSummary;
}

// 获取电影类型
public MovieType getType() {
    return type;
}
```

```
}

// 设置电影类型
public void setType(MovieType type) {
    this.type = type;
}

// 获取电影地区
public Region getRegion() {
    return region;
}

// 设置电影地区
public void setRegion(Region region) {
    this.region = region;
}

// 获取电影导演
public Director getDirector() {
    return director;
}

// 设置电影导演
public void setDirector(Director director) {
    this.director = director;
}

// 获取电影演员列表
public List<Actor> getActors() {
    return actors;
}

// 设置电影演员列表
public void setActors(List<Actor> actors) {
    this.actors = actors;
}

// toString 方法，用于将电影信息转换为字符串
public String toString() {
    StringBuilder sb = new StringBuilder();
    sb.append("Title: ").append(title).append("\n"); // 标题
    sb.append("Release Date: ").append(releaseDate).append("\n"); // 发布日期
    sb.append("Duration: ").append(duration).append(" minutes\n"); // 持续时间
    sb.append("Rating: ").append(rating).append("\n"); // 评分
}
```

```

        sb.append("Plot Summary: ").append(plotSummary).append("\n"); // 情节摘要

        if (type != null) sb.append("Type: ").append(type.getTypeName()).append("\n"); // 类
    型

        if (region != null) sb.append("Region: ").append(region.getRegionName()).append("\n");
    // 地区

        if (director != null) sb.append("Director:
    ").append(director.getDirectorName()).append("\n"); // 导演

        if (actors != null && !actors.isEmpty()) {
            sb.append("Actors: ");
            for (Actor actor : actors) {
                sb.append(actor.getActorName()).append(", ");
            }
            sb.delete(sb.length() - 2, sb.length()); // 删除最后的逗号和空格
            sb.append("\n");
        }

        return sb.toString();
    }

    // 设置电影类型 ID 的方法
    public void setTypeId(int typeId) {
        if (this.type == null) {
            this.type = new MovieType();
        }
        this.type.setTypeId(typeId);
    }

    // 设置电影地区 ID 的方法
    public void setRegionId(int regionId) {
        if (this.region == null) {
            this.region = new Region();
        }
        this.region.setRegionId(regionId);
    }

    // 设置电影导演 ID 的方法
    public void setDirectorId(int directorId) {
        if (this.director == null) {
            this.director = new Director();
        }
        this.director.setDirectorId(directorId);
    }
}

```

六、 总结

项目“面向电影编程”是一个基于面向对象程序设计的电影推荐和查询系统，通过 Java 语言和 MySQL 数据库实现了包括用户管理、电影信息管理、推荐功能等多个模块。以下是对该项目的总结：

1. 项目背景和目的

电影作为一种普遍的娱乐方式，不仅能够唤起情感共鸣和传递思想，还在文化传播和艺术表达中具有重要地位。本项目旨在利用面向对象程序设计的优势，构建一个精确、个性化的电影推荐和查询系统，为用户提供全面且便捷的电影信息获取和交互体验。

2. 技术实现与工具

- 开发工具和环境：** 使用了 MySQL 作为主要的数据存储系统，Navicat Premium 用于数据库管理和开发，Java SE 17 作为开发和运行环境，Eclipse IDE 2024.3 作为集成开发环境，并使用了 WindowBuilder 插件进行 Java GUI 设计。
- 相关技术：** 利用了 Java Swing 库构建图形用户界面（GUI），JDBC 进行与 MySQL 数据库的交互，以及各种设计模式如 DAO 模式、工厂模式等来提高代码的重用性和可维护性。

3. 系统设计与实现

- 数据库设计：** 设计了包括电影、演员、导演等多个表，通过表间的关联和继承建立了完整的数据模型。
- 模型类设计：** 每个核心实体如电影、演员、导演等都有对应的 Java 类，通过面向对象的方法管理各自的属性和方法。
- 数据访问类设计（DAO）：** 采用 DAO 模式，将数据访问逻辑与业务逻辑分离，提供了对数据库的增删改查等操作。
- 工具类设计：** 提供了 DBUtil 类简化数据库连接管理，StringUtil 类处理常用的字符串操作，增强了代码的可维护性和复用性。

- **界面类设计：** 使用了 Swing 库构建了用户友好的图形用户界面，如登录页面、注册页面、用户主页面和管理员主页面等。

4. 功能实现与特点

- **用户管理功能：** 实现了用户的登录、注册功能，区分了普通用户和管理员用户，每种用户类型有不同的权限和操作界面。
- **电影信息管理功能：** 包括电影的添加、查询、更新和删除等操作，管理员可以管理电影、演员和导演的信息。
- **电影推荐功能：** 提供了每日推荐和近期热门推荐的电影功能，根据用户的浏览历史和评分进行智能推荐。
- **界面友好性和交互性：** 通过合理的界面设计和输入验证，提升了用户的操作体验和系统的健壮性。

5. 挑战与解决方案

- **数据库操作的复杂性：** 通过封装 BaseDao 类和具体的 DAO 类简化了与 MySQL 数据库的交互，提高了代码的可维护性。
- **界面设计和用户体验：** 使用 Swing 库和布局管理器设计了直观清晰的界面，确保用户可以轻松理解和使用系统的各项功能。
- **异常处理和数据验证：** 在输入验证和数据库操作中，采用了异常处理机制和数据格式验证，确保系统的稳定性和安全性。

6. 结论与展望

通过本项目的设计与实现，深入理解了面向对象程序设计的优势和应用场景，掌握了 Java 语言、Swing GUI 和与 MySQL 数据库交互的关键技术。未来可以进一步完善系统的推荐算法、优化用户交互体验，并考虑将系统扩展到更多平台和用户群体，提升电影文化的普及和推广。

总体而言，项目“面向电影编程”不仅仅是一个电影信息的存储和检索平台，更是一个通过现代程序设计理念实现的电影文化推广和体验平台，为用户提供了全新的电影探索和欣赏方式。